AAI3008
Large Language Models
Tokenization and Word Embedding

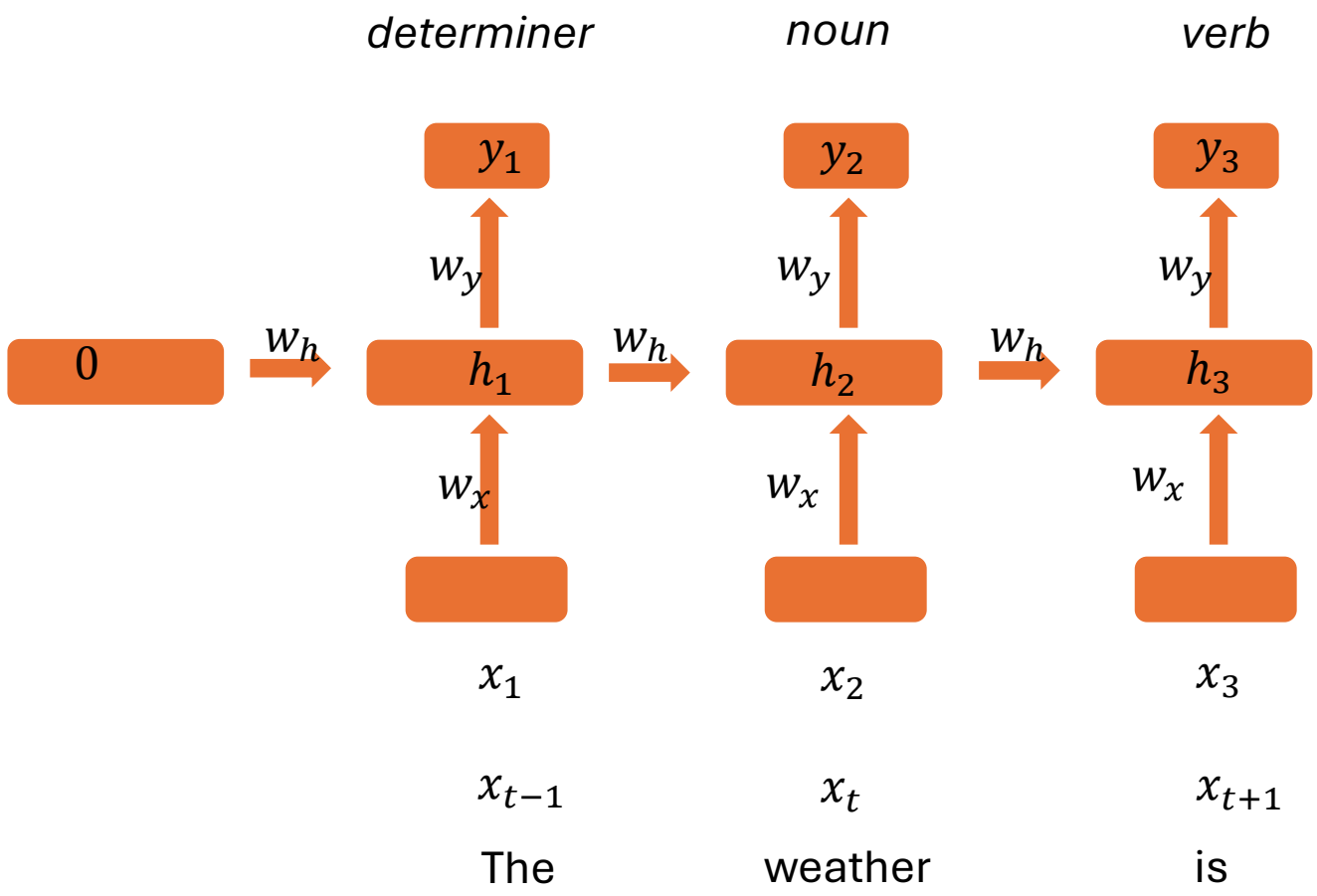
Zhang Zhengchen
zhengchen.zhang@singaporetech.edu.sg

Review

- NLP Tasks
- Word
  - POS Tagging
  - Stemming and Lemmatization
- Sentence
  - Sentiment Analysis
  - Syntactic Parsing
  - Text Generation
  - Named Entity Recognition (NER)
- Document
  - Text Summarization
  - Document Classification (e.g., News Sentiment / Topic)
- Corpus-Level
  - Language Modelling: N-grams
  - Collocation Extraction

Review-RNN

- Sequential Data
- Time Step t
  - A word
  - A segment of audio
  - Stock price of a day
- Hidden state h_t depends on the current input (x_t) and the previous state ($\boldsymbol{h}_(\boldsymbol{t}-\mathbf{1})$)
- $h_t = f(W_h h_{t-1} + W_x x_t + b)$
- h_t stores information from current and past time steps

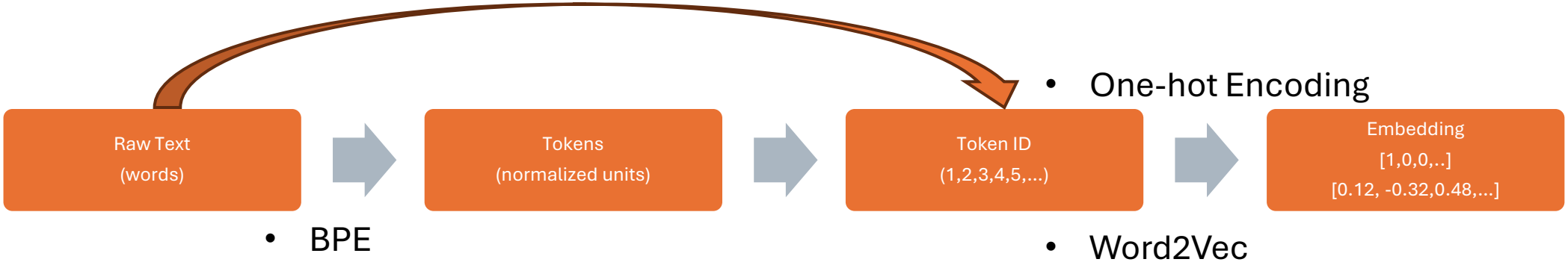


Tokenization

- The process of dividing a text into smaller units known as tokens.
- Sub-word matters
  - Run, runs, running
  - Fair, fairness
- Tokens
  - Words or sub-words in the context of natural language processing.

Outline

- From Raw Text to Neural Network Input



Byte Pair Encoding (BPE)

- A sub-word tokenization algorithm that **builds sub-words** from characters by **merging frequent pairs**
  - Represent rare words as combinations of frequent sub-words.

- Sub-word matters: from a Machine Translation Perspective
  - Various word classes are translatable via smaller units than words
    - *names (via character copying or transliteration)*
    - *compounds (via compositional translation)*
    - *cognates and loanwords (via phonological and morphological transformations)*[1]

1. Sennrich R. Neural machine translation of **rare words** with subword units[J]. arXiv preprint arXiv:1508.07909, 2015. https://arxiv.org/pdf/1508.07909

Byte Pair Encoding (BPE)

- A sub-word tokenization **algorithm** that builds **sub-words** from characters *by merging frequent pairs*

- Input
  - A set of words
- Output
  - A set of sub-words (tokens)
  - Merge rules

Byte Pair Encoding (BPE)

Algorithm 1 Learn BPE operations

```
import re, collections

def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
    for i in range(len(symbols)-1):
      pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
         'n e w e s t </w>':6, 'w i d e s t </w>':3}
num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
  print(best)
```

1. Split word into characters, form a **vocabulary**
   - low → l o w </w>
   - 'l o w </w>' : 5
2. Count the frequency of each bigram, get **pair statistics**
   - Tow characters:
   - 'l o' : 7; 'o w' : 7
   - A sub-word and a character: lo w
3. Merge the most frequently appeared bigram
   - l o → lo
   - 'lo w </w>' : 5
4. Repeat 2 and 3 until some threshold is met
   - Number of merges
   - Vocabulary size

Sennrich R. Neural machine translation of rare words with subword units[J]. arXiv preprint arXiv:1508.07909, 2015.
https://www.pure.ed.ac.uk/ws/portalfiles/portal/174256085/Neural_Machine_Translation_SENNRICH_DOA24052016_VOR_CC_BY.pdf

Byte Pair Encoding (BPE)

- Output
  - A set of sub-words
  - Merge rules (order is important)
- new sub-word dictionary (token list):
  - {'est</w>', '</w>', 'e', 'wi', 'low', 'd', 'low</w>', 'r', 'newest</w>'}
- Merge Rules
  - [('e', 's'), ('es', 't'), ('est', '</w>'), ('l', 'o'), ('lo', 'w'), ('n', 'e'), ('ne', 'w'), ('new', 'est</w>'), ('low', '</w>'), ('w', 'i')]

Tokenize New Text

- Convert words in new text into subwords.
- Apply each learned merge rule in the order they were learned.
- Any token not in the vocabulary is replaced with the unknown token “[UNK]”.

- Cache tokenization results for words seen in the training set.

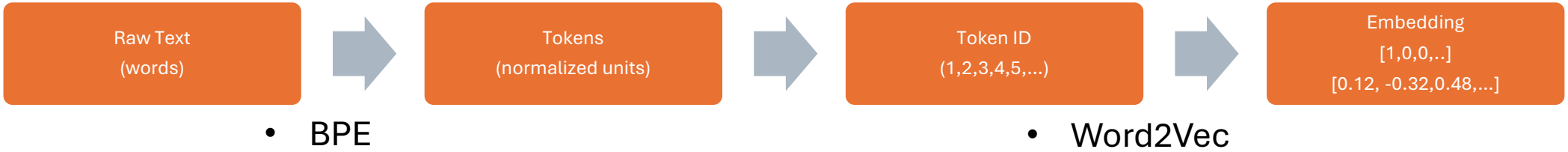
https://medium.com/@hsinhungw/understanding-byte-pair-encoding-fd196ebfe93f

Tokenizers in real world

- Input
  - paper:
    - A set of words
    - Word to character sequences
    - Count
  - GPT2
    - A sequence of characters, including whitespace
    - Whitespace is treated as a character
    - Count
- Output
  - A set of tokens
  - Merge rules

Outline

- Concepts about Text Processing
- BPE
- **Word Embedding**



Word Embedding

- Why Do We Need Word Embeddings?
  - Efficient Representation
    - Dense vectors (e.g., 100-300 dimensions) reduce memory usage
  - Capture Semantic Relationships
    - One-hot Embeddings are just random IDs
    - For example: happy/glad
    - Similar words have similar vector representations
      - Cosine Similarity

How Are Embeddings Learned?

- Pre-trained Embeddings
  - Learned in advance on large datasets
  - Reused in downstream tasks
- Trainable Embeddings
  - Randomly initialized
  - Learned together with the task
  - e.g. nn.Embedding (used in our labs)

Word2Vec

- Pre-trained Embeddings
- Learns vector representations of words using a neural network
  - Continuous Bag of Words (CBOW)
    - Predicts a target word from its surrounding context.
  - Skip-Gram
    - Predicts surrounding words given a target word.

Mikolov T. Efficient estimation of word representations in vector space[J]. arXiv preprint arXiv:1301.3781, 2013, 3781. https://arxiv.org/pdf/1301.3781

Continuous Bag of Words (CBOW)

- What is Bag of Words?
  - A **text document** is represented as a vector of word counts by **ignoring word order and position**, while retaining only the frequency of each word.

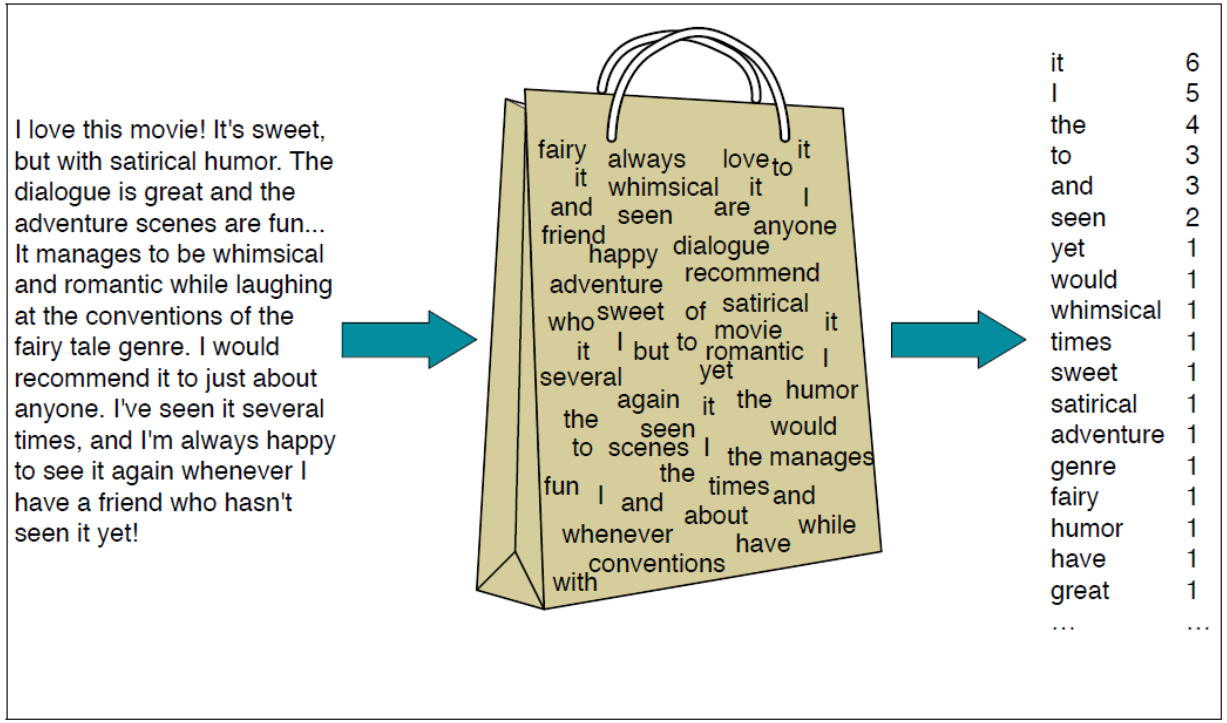


Figure 4.1 Intuition of the multinomial naive Bayes classifier applied to a movie review. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Chapter 4, https://web.stanford.edu/~jurafsky/slp3/

Bag of Words

- Today's weather is not bad.
  - [1,1,1,1,1,1,0]
- The weather is bad.
  - [0,0,1,1,0,1,1]

	Word	One-hot Encoding
1	Today	[1,0,0,0,0,0,0]
2	's	[0,1,0,0,0,0,0]
3	Weather	[0,0,1,0,0,0,0]
4	Is	[0,0,0,1,0,0,0]
5	Not	[0,0,0,0,1,0,0]
6	Bad	[0,0,0,0,0,1,0]
7	the	[0,0,0,0,0,0,1]

Continuous Bag of Words (CBOW)

- Represent words in a continuous vector space
- Predicts a target word from its surrounding context.
- Output
  - Vector of a target word
- Input
  - Context
  - The words around the target word
  - Window size

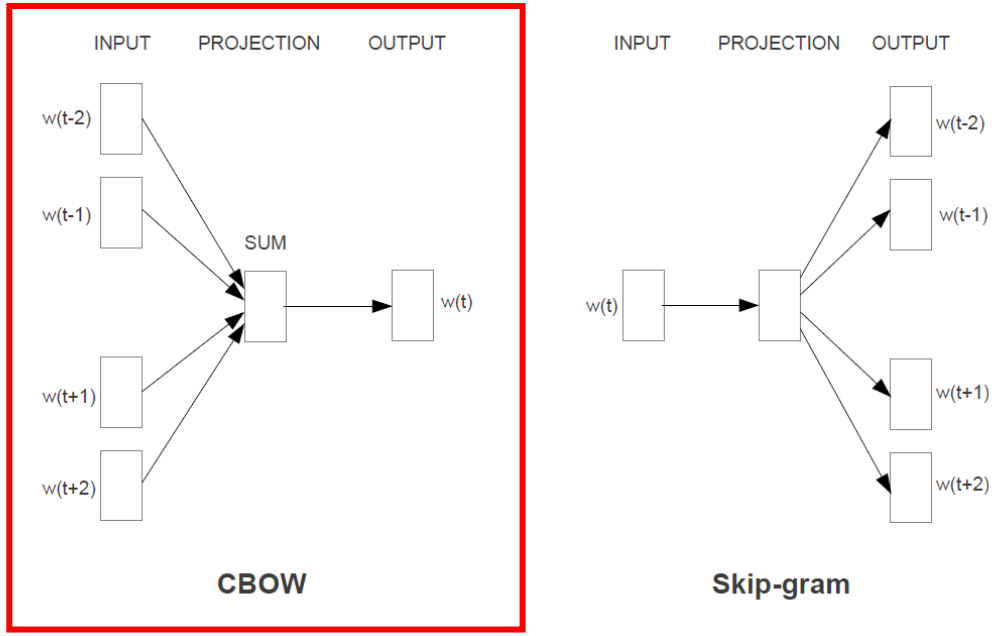


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Mikolov T. Efficient estimation of word representations in vector space[J]. arXiv preprint arXiv:1301.3781, 2013, 3781. https://arxiv.org/pdf/1301.3781

Continuous Bag of Words (CBOW)

- "The cat sat on the mat."
- Output
  - Vector of the target word
  - "sat", $w(t)$
- Input
  - Context words within a fixed-size window around the target word
  - Context words (window size = 2)
    - ["The", "cat", "on", "the"]
    - $w(t-2), w(t-1), w(t+1), w(t+2)$
- Each word is represented as a one-hot encoded vector of size V (vocabulary size).

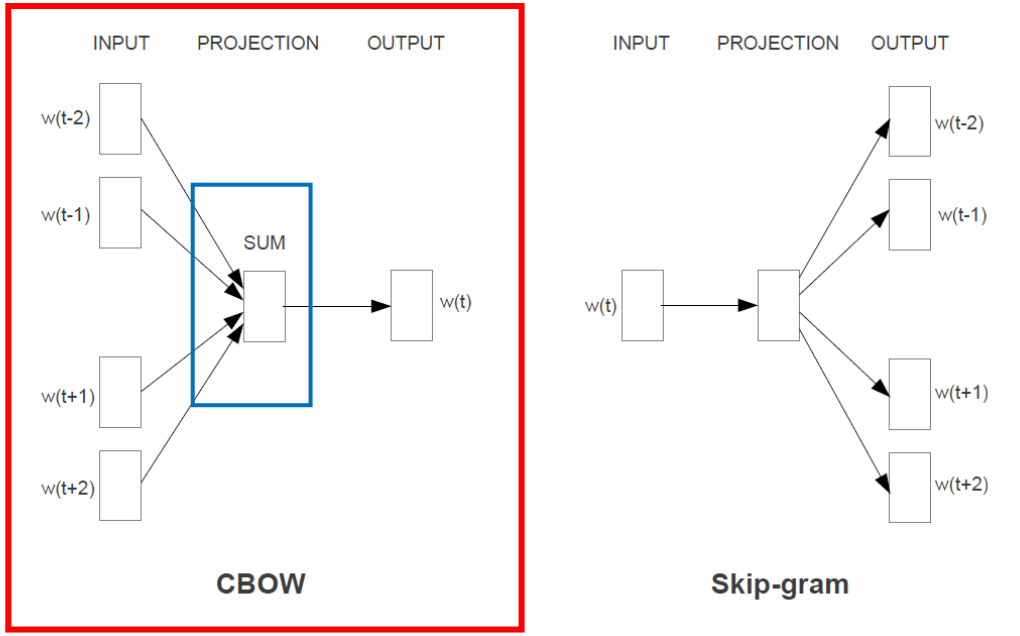
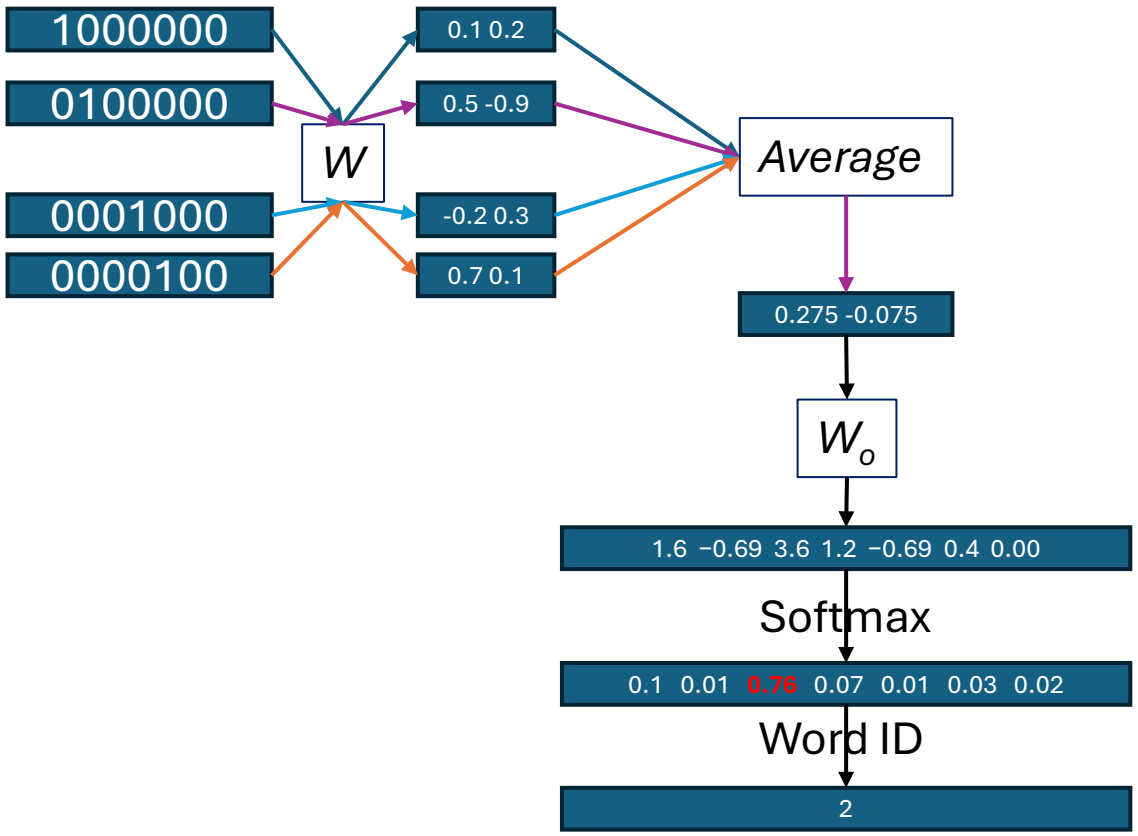


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Mikolov T. Efficient estimation of word representations in vector space[J]. arXiv preprint arXiv:1301.3781, 2013, 3781. https://arxiv.org/pdf/1301.3781

Continuous Bag of Words (CBOW)

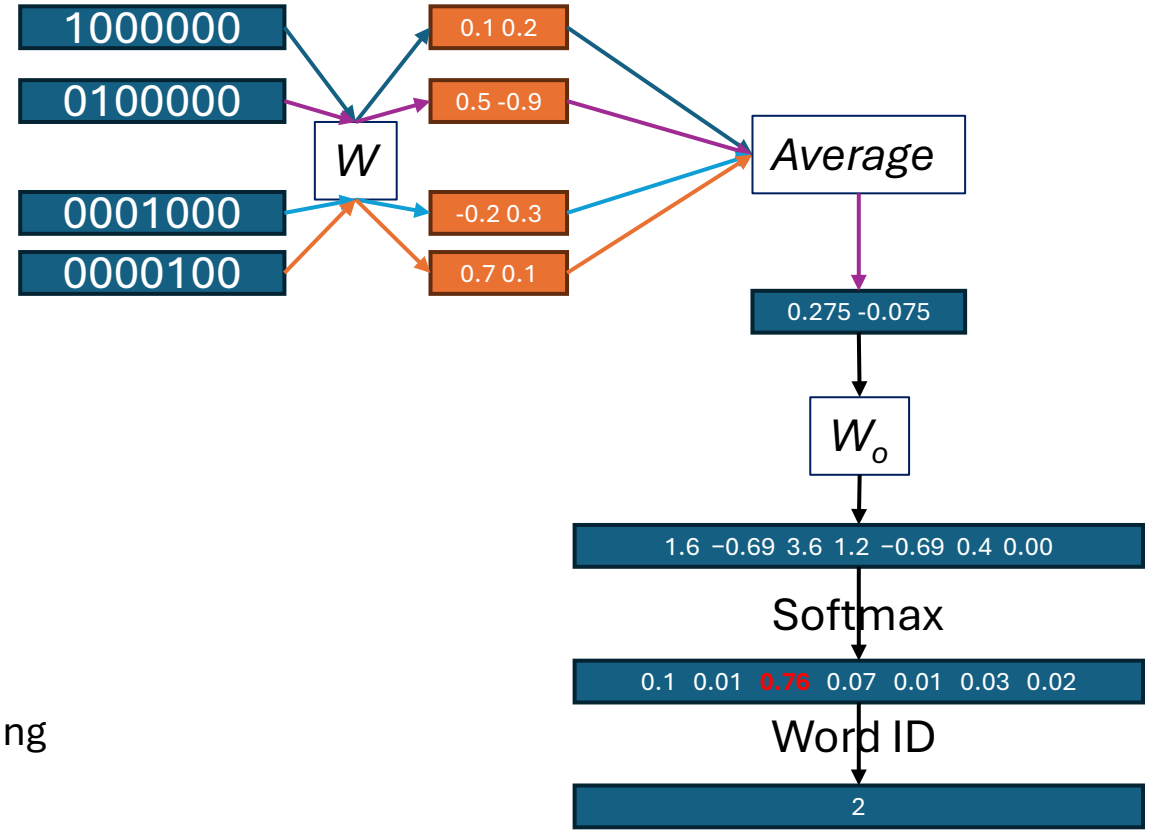
- Project one-hot vector to a dense, **low-dimensional** embedding space.
- W is a $V * N$ matrix
  - V is the vocabulary size
  - N is the embedding dimension (e.g., 100 or 300).
- Embedding vector for a word x_i
  - $h_i = W^T x_i$
- Averaging Embeddings
  - $h = \frac{1}{C}\sum_{i=1}^{C} h_i$
  - C is the number of context words.
- Output word $o = W_o^T h$
  - Size of W_o is $N * V$
  - o is a vector of size V
- Softmax Function $\mathrm{P}(w_t | w_{t_{context}}) = \frac{e^{o_i}}{\sum_{j=1}^{V} e^{o_j}}$
- The model predicts the word with the highest probability



Continuous Bag of Words (CBOW)

- Loss Function
  - cross-entropy loss
  - $L = -\sum_{i=1}^{V} y_i \log(\hat{y}_i)$
  - $\hat{y}_i = \frac{e^{o_i}}{\sum_{j=1}^{V} e^{o_j}}$
- If the true label y is one hot embedding
  - $y_i = 1$
  - $y_{j \neq i} = 0$
  - $L = -\log \hat{y}_i$

- Inference
  - We take $h_i = W^T x_i$ of word i as its word embedding



https://ml-cheatsheet.readthedocs.io/en/latest/loss_functions.html

An Engineering Insight

- One-Hot × Weight Matrix = Embedding Lookup
- $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0.2 & 0.5 \\ 1.3 & -0.7 \\ 0.1 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.2 & 0.5 \\ 1.3 & -0.7 \\ 0.1 & 0.9 \end{bmatrix}$

- identity matrix

Gensim

- Gensim is a free open-source Python library for representing documents as semantic vectors, as efficiently (computer-wise) and painlessly (human-wise) as possible.
- https://radimrehurek.com/gensim/index.html

```
import gensim.downloader
print(list(gensim.downloader.info()['models'].keys()))
# ['fasttext-wiki-news-subwords-300', 'conceptnet-numberbatch-17-06-300', 'word2vec-ruscorpora-300', 'word2vec-google-news-300', 'glove-wiki-gigaword-50', 'glove-wiki-gigaword-100', 'glove-wiki-gigaword-200', 'glove-wiki-gigaword-300', 'glove-twitter-25', 'glove-twitter-50', 'glove-twitter-100', 'glove-twitter-200', '__testing_word2vec-matrix-synopsis']
google_news = gensim.downloader.load('word2vec-google-news-300')
#[==================================================] 100.0% 1662.8/1662.8MB downloaded
words = google_news.most_similar('happy')
print(words)
#[('glad', 0.7408890724182129), ('pleased', 0.6632170677185059), ('ecstatic', 0.6626912355422974), ('overjoyed', 0.6599286794662476), ('thrilled', 0.6514049172401428), ('satisfied', 0.6437949538230896), ('proud', 0.636042058467865), ('delighted', 0.627237856388092), ('disappointed', 0.6269949674606323), ('excited', 0.6247665286064148)]
king_man_vector = google_news['king'] - google_news['man']
queen_woman_vector = google_news['queen'] - google_news['woman']
similarity_vectors = google_news.cosine_similarities(king_man_vector, [queen_woman_vector])
print(f"Cosine similarity between 'king - man' and 'queen - woman': {similarity_vectors[0]:.4f}")
# Cosine similarity between 'king - man' and 'queen - woman': 0.7580
```

Cosine Similarity

- Measures semantic similarity between vectors
- Tells us whether two representations point in a similar direction
- Focuses on direction (angle) rather than length

Cosine Similarity

- Dot Product / Inner Product

$$dot\ product(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \cdots + v_N w_N$$

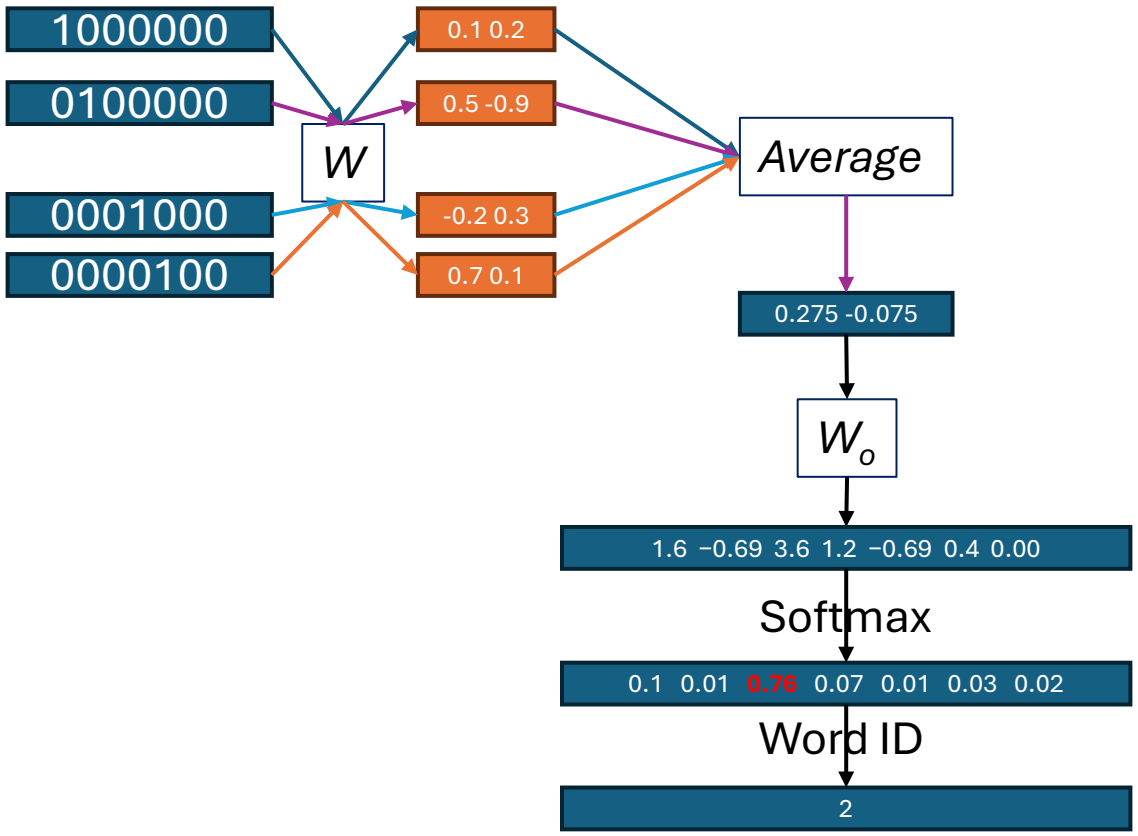
- Vector Length

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

- Cosine Similarity

$$cosine(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}||\mathbf{w}|}$$

Review - Word2Vec



Skip-Gram

- The model will try to **predict each context word** given the target word.
- "The cat sat on the mat."
- Target word w_t='sat'
  - Window Size (N=2)
  - Context words: $\{w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}\}$
  - {"the", "cat", "on", "the"}
- Compose training Pairs
  - ("sat","the")
  - ("sat","cat")
  - ("sat","on")
  - ("sat","the")

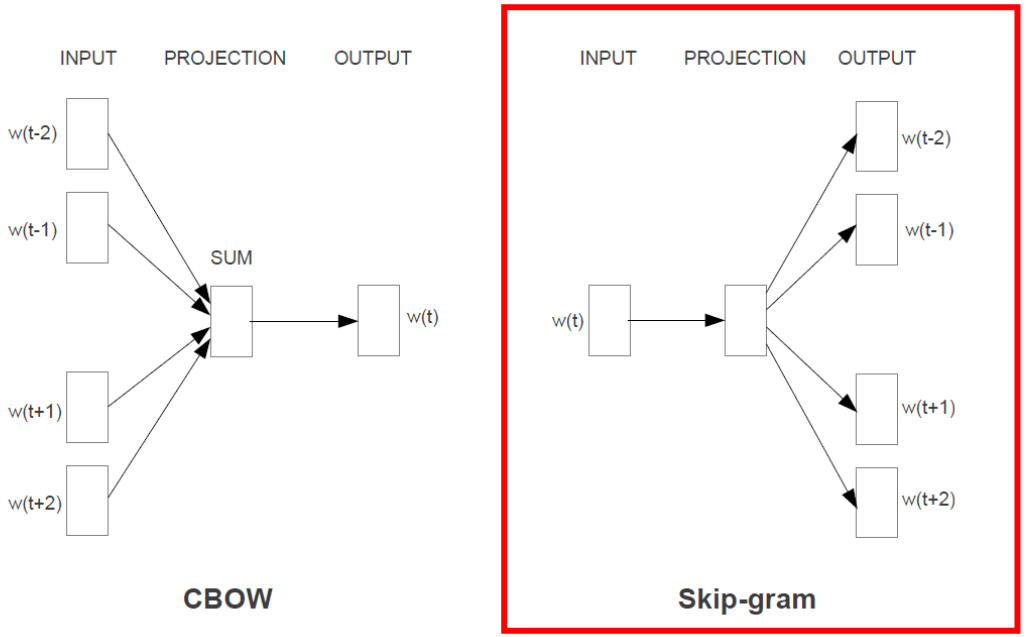


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Skip-Gram

- Input
  - "sat"
- Output
  - "cat"
  - "on"
  - "the"

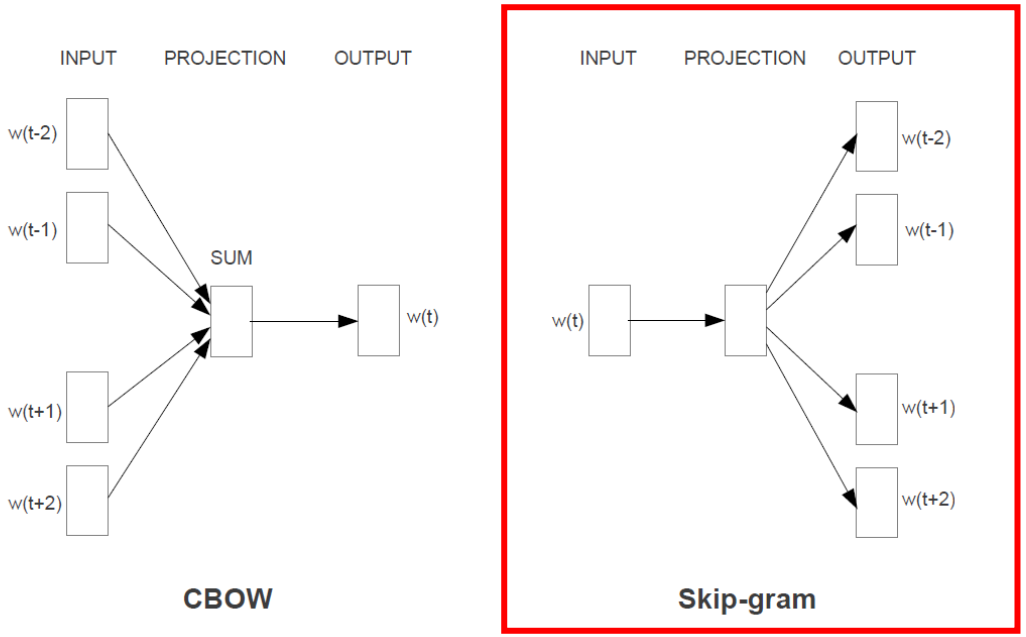
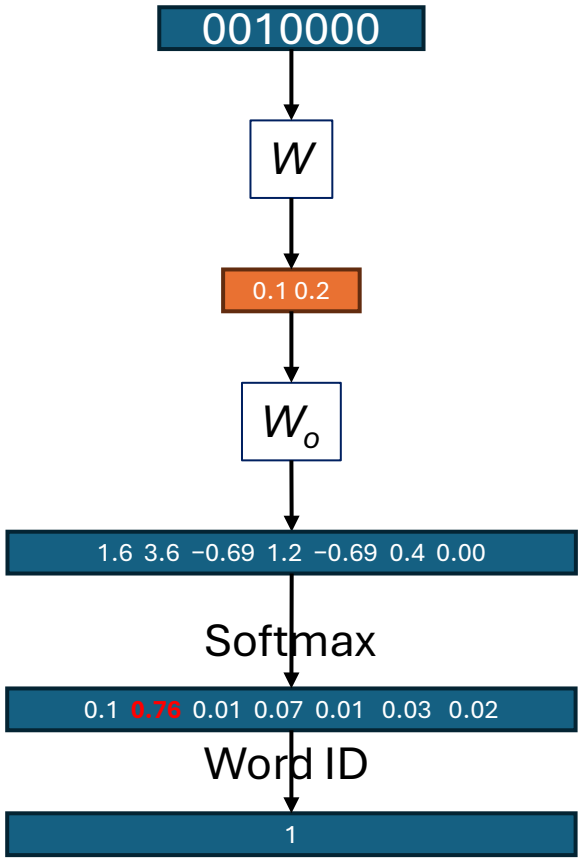


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Skip-Gram

- Project one-hot vector to a dense, **low-dimensional** embedding space.
- W is a $V * N$ matrix
  - V is the vocabulary size
  - N is the embedding dimension (e.g., 100 or 300).
- Embedding vector for a word w_t
  - $h_t = W^T x_t$
- Output word $o = W_o^T h_t$
  - Size of W_o is $N * V$
  - o is a vector of size V
- Softmax Function $\mathrm{P}(w_\mathrm{i}|w_\mathrm{t}) = \frac{e^{o_i}}{\sum_{j=1}^{V} e^{o_j}}$
- The model predicts the word with the highest probability



Skip-Gram

- Discussion
  - One input, multiple true output
    - The training target is to get the vectors
    - Not the one-to-one mapping
  - Reduce the computational complexity
    - Negative Sampling*
      - Focuses on a small subset of "negative" samples instead of the full vocabulary
    - Hierarchical Softmax*
      - Reduces computational complexity by structuring the vocabulary as a binary tree
      - Huffman Tree*
        - Huffman tree is built by repeatedly merging the two least frequent nodes.
        - Assign shorter paths to more frequent words, making training even more efficient.

Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality[J]. Advances in neural information processing systems, 2013, 26.
https://arxiv.org/pdf/1310.4546

Learning outcomes

- Gain an understanding of the **BPE** (Byte Pair Encoding) algorithm.
  - Build foundational knowledge to comprehend more complex models, such as BERT and GPT.
- Understand the principles and **mechanisms** behind **Word2Vec**.